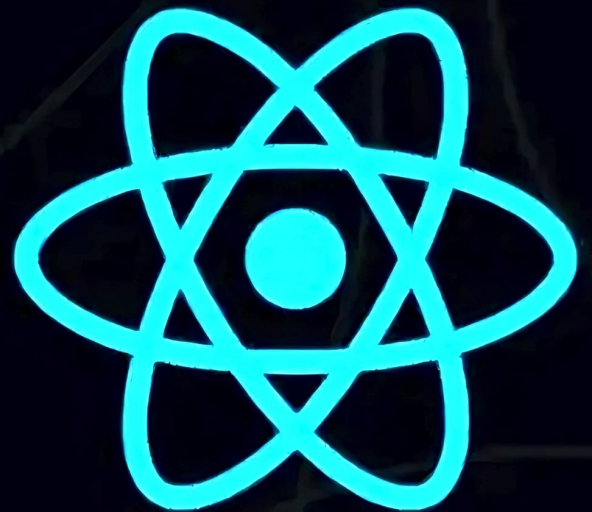


State in React





State in React is an **object** that holds information that may change over the time of a component.

State is managed **within the component** (similar to variables declared inside a function). When the state of a component changes, React **re-renders that component** to reflect the changes.

Syntax :

```
const [state, setState] = useState(initialState);
```

useState :

useState is a **React hook** that allows you to add **state to a functional component**.

It takes one argument, the **initial state** (initialState), and returns an **array with two elements**.

initialState :

useState is a **React hook** that allows you to add **state to a functional component**.

It takes one argument, the **initial state** (initialState), and returns an **array with two elements**.

```
const [state, setState] = useState(initialState);
```

Destructuring the Array :

The array returned by `useState` contains exactly two elements:

- 1.The current state value.
- 2.A function to update the state.

`state` : will hold the current state value.

`setState` : is the function used to update the state.

Example :



```
import React, { useState } from 'react';

const Counter = () => {
  // Initialize state with 0
  const [count, setCount] = useState(0);

  // Function to update the state
  const increment = () => {
    setCount(count + 1);
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick=
{increment}>Increment</button>
    </div>
  );
}

export default Counter;
```

Explanation :

Initialization :

```
const [count, setCount] = useState(0);
```

- `useState(0)` initializes the state to 0.
- `count` is the current state value (initially 0).
- `setCount` is the function used to update the state.

Rendering :

```
return (  
  <div>  
    <p>Count: {count}</p>  
    <button onClick={increment}>Increment</button>  
  </div>  
);
```

- The component renders the current value of `count` inside a `<p>` tag.
- A button is rendered with an `onClick` event handler that calls the `increment` function when clicked

Updating State :

```
const increment = () => {  
  setCount(count + 1);  
}
```

- The increment function calls `setCount(count + 1)`.
- `setCount` updates the state with the new value `(count + 1)`.
- When the state updates, `React` re-renders the component with the new state value.

Summary:

- `useState` hook allows you to add state to functional components.
- It returns an array with two elements: the current state and a function to update the state.
- You use array destructuring to extract these two elements into `state` and `setState`.
- `state` holds the current value of the state, and `setState` is used to update the state.
- When `setState` is called, the component re-renders with the updated state.